MTRC

MODERN INSTITUTE OF TECHNOLOGY AND RESEARCH CENTRE

Name of Faculty: MOHIT SHARMA

Subject & Code: ANALYSIS OF ALGORITHM & 6CS4-02

Branch: CSE 6TH SEM Batch: A&B

Semester: VIth **Session:** 2023-24(even Sem.)

Unit No.: 5 **Date of Submission: 09-04-2024**

Lect.No.	Topics Covered	Page No.	Total pages
1	Computational Complexity	2-5	4
2	Polynomial Vs non-polynomial time complexity	6-7	2
3	NP-hard and NP-complete classes	8-10	3
4	Examples: Circuit Satisfiablity	11-12	2
5	Vertex cover, Subset Sum problem	13-16	4
6	Randomized Algorithms, String Matching	17-21	5
7	NP-Hard and NP Completeness, Introduction to Approximation Algorithms	22-23	2

References Book:

1. T .H .Cormen, C .E .Leiserson, R .L . Rivest "Introduction to Algorithms", 3rd Ed.,PHI, 2011
2. **E. Horowitz, S. Sahni, and S. Rajsekaran, "Fundamentals of Computer Algorithms,"Galgotia Publication**
3. **S.K Basu- Design Methods and Analysis of Algorithms, 2nd Ed., PHI**

FACULTY HOD DEAN ACADEMICS

UNIT 5

Lecture 1

Introduction to Computational Complexity Theory

Overview

Computational complexity theory studies the complexity of algorithms and problems. It aims to identify the computational resources required to solve a problem, such as time, space, or communication, and to determine the limitations and possibilities of algorithmic efficiency.

One of the most important concepts in computational complexity theory is the P versus NP problem, which asks whether every problem that can be verified in polynomial time can also be solved in polynomial time. The class P consists of problems that can be solved in polynomial time, while the class NP consists of problems for which a solution can be verified in polynomial time.

NP-complete problems are a special subset of NP problems that are believed to be among the hardest problems in the class, in the sense that any NP-complete problem can be reduced to any other NP-complete problem in polynomial time. The notion of NP-completeness has important implications in cryptography, optimization, and many other areas of computer science and mathematics.

Other important concepts in computational complexity theory include the notion of polynomial-time reductions, which allows the complexity of one problem to be compared to another, and the complexity classes beyond P and NP, such as PSPACE and EXPTIME.

Overall, computational complexity theory is a rich and important field of study that provides a framework for understanding the computational limits and possibilities of algorithms and problems.

Solvable vs. Unsolvable Problems

In computational complexity theory, problems are often classified as either solvable or unsolvable, based on whether there exists an algorithm that can solve the problem or not.

1. **Solvable Problem** - A problem is said to be solvable if there exists an algorithm that can solve it in a finite amount of time, i.e., for any input of the problem, the algorithm can produce the correct output in a finite

number of steps. For example, the problem of finding the sum of two integers is solvable, since there exists a simple algorithm that can add two integers in a finite number of steps.

2. **Unsolvable Problem** - On the other hand, a problem is said to be unsolvable if there does not exist any algorithm that can solve it in a finite amount of time. One well-known example of an unsolvable problem is the Halting problem, which asks whether a given program will eventually halt or run forever. It has been proven that there is no algorithm that can solve the Halting problem for all possible inputs.

It is important to note that unsolvable problems are not the same as difficult problems. Some problems may be solvable, but require an exponential amount of time or space to solve, making them practically infeasible for large input sizes. The study of computational complexity theory focuses on understanding the limits of algorithmic efficiency, and identifying which problems are solvable, which problems are unsolvable, and which problems are solvable but infeasible.

Decidable vs. Undecidable Problems

In computational complexity theory, solvable problems are also often classified as decidable or undecidable, based on whether there exists an algorithm that can decide the problem or not.

a. **Decidable Problem** - A problem is said to be decidable if there exists an algorithm that can determine whether an instance of the problem is a "yes" instance or a "no" instance in a finite amount of time. In other words, the algorithm will always halt and return the correct answer, either "yes" or "no". For example, the problem of determining whether a given number is prime is decidable, since there exists an algorithm that can determine the primality of a number in a finite amount of time.

b. **Undecidable Problem** - On the other hand, a problem is said to be undecidable if there is no algorithm that can determine whether an instance of the problem is a "yes" instance or a "no" instance in a finite

amount of time. The classic example of an undecidable problem is the Halting problem, which asks whether a given program will eventually halt or run forever. It has been proven that there is no algorithm that can solve the Halting problem for all possible inputs.

It is worth noting that undecidability is a stronger notion than insolvability. All undecidable problems are unsolvable, but not all unsolvable problems are undecidable. For example, the problem of finding an optimal solution to the traveling salesman problem is unsolvable for large input sizes, but it is still decidable, since there exist algorithms that can find a suboptimal solution in a finite amount of time.

The study of decidable and undecidable problems is important for understanding the limits of algorithmic solvability, and for identifying the scope and limits of automated decision-making in computer science and other fields.

P vs. NP Problems

In computational complexity theory, decidable problems are also often classified as P or NP problems.

The P versus NP problem is one of the most famous open problems in computer science and mathematics. It asks whether every problem that can be verified in polynomial time can also be solved in polynomial time. In other words, if a solution to a problem can be checked efficiently (in polynomial time), can it also be found efficiently (in polynomial time)?

The class of problems that can be solved in polynomial time is known as P, while the class of problems that can be verified in polynomial time is known as NP. Many important computational problems are known to be in NP, including the traveling salesman problem, the knapsack problem, and the satisfiability problem.

The P versus NP problem has important implications for cryptography, optimization, and other fields of computer science and mathematics. If P = NP, then it would be possible to solve many important computational problems much more efficiently than is currently known, and this could have major implications for fields such as cryptography and data analysis. However, if P ? NP, then many important computational problems may be fundamentally intractable, and this could have important implications for the limits of automated decision-making.

Despite decades of research, the P versus NP problem remains one of the most challenging open problems in computer science and mathematics, and it is not yet clear whether it will be resolved in the near future.

What are the major problems in Computational Complexity?

There are several major problems in computational complexity theory that remain open and actively researched. Here are a few examples:

1. **P versus NP problem:** As I mentioned earlier, this is one of the most famous open problems in computer science. It asks whether every problem that can be verified in polynomial time can also be solved in polynomial time.
2. **Complexity of specific problems**: There are many important computational problems for which the complexity is not yet well understood. Examples include the traveling salesman problem, graph isomorphism, and factoring large integers.
3. **Circuit lower bounds:** Despite many efforts, there is still no known general technique for proving lower bounds on the size of Boolean circuits that compute specific functions. This is a major open problem in computational complexity theory.
4. **De-randomization:** Randomized algorithms are often more efficient than deterministic algorithms, but it is not known whether every randomized algorithm can be converted into an equally efficient deterministic algorithm.
5. **Quantum complexity theory:** Quantum computers have the potential to solve certain problems much faster than classical computers, but the study of quantum complexity theory is still in its early stages.

These are just a few examples of the many open problems in computational complexity theory. Despite their difficulty, progress in these areas can have significant implications for cryptography, optimization, machine learning, and other fields of computer science and mathematics.

Lecture 2

Difference between P class Problem and NP class Problem

In this article, we will understand the differences between the P class Problem and the NP class Problem.

P Class Problem:

A P class problem can be solved in "polynomial time," which means that an algorithm exists for its solution such that the number of steps in the algorithm is bounded by a polynomial function of n, where n corresponds to the length of the input for the problem. This problem is easy to understand and tractable.

NP Class Problem:

Sr. No	P class Problem	NP class Problem
1.	P problems are a set of problems that can be solved in polynomial time by deterministic algorithms.	NP problems are problems that can be solved nondeterministic polynomial time.
2.	P Problems can be solved and verified in polynomial time.	The solution to NP problems cannot be obtained polynomial time, but if the solution is given, it can verified in polynomial time.
3.	P problems are a subset of NP problems.	NP Problems are a superset of P problems.
4.	All P problems are deterministic in nature.	All the NP problems are non-deterministic in nature.
5.	It takes polynomial time to solve a problem like n, n^2, n*logn, etc.	It takes non-deterministic polynomial time to quic check a problem.
6.	The solution to P class problems is easy to	The solution to NP class problems is hard to find.

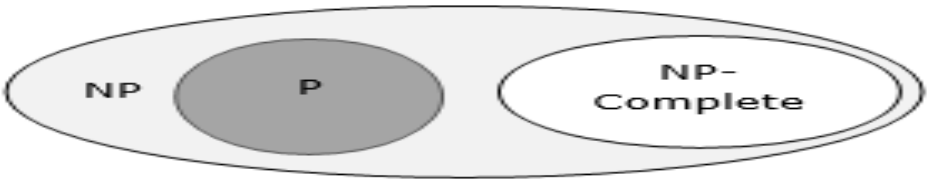
	find.	
7.	**Examples of P problems are:** Selection sort, Linear Search	**Examples of NP problems are** the Travelling sales problem and the knapsack problem.

A problem is said to be Nondeterministic polynomial time that can be solvable in polynomial time by a nondeterministic Turing machine. The solutions to the NP class problem are hard to find since they are being solved by a non-deterministic machine.

Lecture 3

NP Hard & NP-Complete Classes

A problem is in the class NPC if it is in NP and is as **hard** as any problem in NP. A problem is **NP-hard** if all problems in NP are polynomial time reducible to it, even though it may not be in NP itself.



If a polynomial time algorithm exists for any of these problems, all problems in NP would be polynomial time solvable. These problems are called **NP-complete**. The phenomenon of NP-completeness is important for both theoretical and practical reasons.

Definition of NP-Completeness

A language **B** is ***NP-complete*** if it satisfies two conditions

- **B** is in NP
- Every **A** in NP is polynomial time reducible to **B**.

If a language satisfies the second property, but not necessarily the first one, the language **B** is known as **NP-Hard**. Informally, a search problem **B** is **NP-Hard** if there exists some **NP-Complete** problem **A** that Turing reduces to **B**.

The problem in NP-Hard cannot be solved in polynomial time, until **P = NP**. If a problem is proved to be NPC, there is no need to waste time on trying to find an efficient algorithm for it. Instead, we can focus on design approximation algorithm.

NP-Complete Problems

Following are some NP-Complete problems, for which no polynomial time algorithm is known.

- Determining whether a graph has a Hamiltonian cycle
- Determining whether a Boolean formula is satisfiable, etc.

NP-Hard Problems

The following problems are NP-Hard

- The circuit-satisfiability problem
- Set Cover
- Vertex Cover
- Travelling Salesman Problem

In this context, now we will discuss TSP is NP-Complete

TSP is NP-Complete

The traveling salesman problem consists of a salesman and a set of cities. The salesman has to visit each one of the cities starting from a certain one and returning to the same city. The challenge of the problem is that the traveling salesman wants to minimize the total length of the trip

Proof

To prove ***TSP is NP-Complete***, first we have to prove that ***TSP belongs to NP***. In TSP, we find a tour and check that the tour contains each vertex once. Then the total cost of the edges of the tour is calculated. Finally, we check if the cost is minimum. This can be completed in polynomial time. Thus ***TSP belongs to NP***.

Secondly, we have to prove that ***TSP is NP-hard***. To prove this, one way is to show that ***Hamiltonian cycle*** $\leq_p$ ***TSP*** (as we know that the Hamiltonian cycle problem is NPcomplete).

Assume ***G = (V, E)*** to be an instance of Hamiltonian cycle.

Hence, an instance of TSP is constructed. We create the complete graph $G' = (V, E')$, where

$E'=\{(i,j):i,j\in V and i\neq j$

Thus, the cost function is defined as follows −

$t(i,j)=\{01if(i,j)\in Eotherwise$

Now, suppose that a Hamiltonian cycle ***h*** exists in ***G***. It is clear that the cost of each edge in ***h*** is **0** in G' as each edge belongs to ***E***. Therefore, ***h*** has a cost of **0** in G'. Thus, if graph ***G*** has a Hamiltonian cycle, then graph G' has a tour of **0** cost.

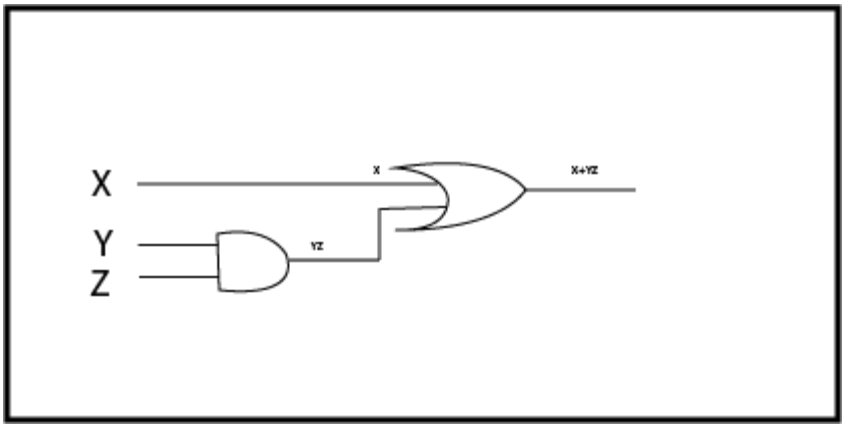
Conversely, we assume that $\boldsymbol{G}'$ has a tour $\boldsymbol{h}'$ of cost at most **0**. The cost of edges in $\boldsymbol{E}'$ are **0** and **1** by definition. Hence, each edge must have a cost of **0** as the cost of $\boldsymbol{h}'$ is **0**. We therefore conclude that $\boldsymbol{h}'$ contains only edges in $\boldsymbol{E}$.

We have thus proven that $\boldsymbol{G}$ has a Hamiltonian cycle, if and only if $\boldsymbol{G}'$ has a tour of cost at most **0**. TSP is NP-complete

Lecture 4

CIRCUIT SAT

According to given decision-based NP problem, you can design the CIRCUIT and verify a given mentioned output also within the P time. The CIRCUIT is provided below:-



Note:- We can design a circuit and verified the mentioned output within Polynomial time but remember you can never predict the number of gates which produces the high output against the set of inputs/high inputs within a polynomial time. So you verified the production and conversion had been done within polynomial time. So you are in NPC.

SAT (Satisfiability):-

A Boolean function is said to be SAT if the output for the given value of the input is true/high/1

F=X+YZ (Created a Boolean function by CIRCUIT SAT)

These points you have to be performed for NPC

1. CONCEPTS OF SAT
2. CIRCUIT SAT$\leq\rho$ SAT
3. SAT$\leq\rho$ CIRCUIT SAT
4. SAT ϵ NPC

1. **CONCEPT:** - A Boolean function is said to be SAT if the output for the given value of the input is true/high/1.
2. **CIRCUIT SAT≤ρ SAT:** - In this conversion, you have to convert CIRCUIT SAT into SAT within the polynomial time as we did it
3. **SAT≤ρ CIRCUIT SAT:** - For the sake of verification of an output you have to convert SAT into CIRCUIT SAT within the polynomial time, and through the CIRCUIT SAT you can get the verification of an output successfully
4. **SAT ϵ NPC:** - As you know very well, you can get the SAT through CIRCUIT SAT that comes from NP.

Proof of NPC: - Reduction has been successfully made within the polynomial time from CIRCUIT SAT TO SAT. Output has also been verified within the polynomial time as you did in the above conversation.

So concluded that SAT ϵ NPC.

Lecture 5

Subset Sum is NP Complete

Prerequisite: NP-Completeness, Subset Sum Problem
Subset Sum Problem: Given **N** non-negative integers **a1…aN** and a target sum **K**, the task is to decide if there is a subset having a sum equal to **K**.
Explanation: An instance of the problem is an input specified to the problem. An instance of the subset sum problem is a set **S = {a1, …, aN}** and an integer **K**. Since an NP-complete problem is a problem which is both in **NP** and **NP-hard**, the proof for the statement that a problem is NP-Complete consists of two parts:

1. *The problem itself is in NP class.*
2. *All other problems in NP class can be polynomial-time reducible to that. (B is polynomial-time reducible to C is denoted as $B \leq PC$)*

If the **2nd condition** is only satisfied then the problem is called **NP-Hard**.
But it is not possible to reduce every NP problem into another NP problem to show its NP-Completeness all the time. That is why if we want to show a problem is NP-Complete we just show that the problem is in NP and any NP-Complete problem is reducible to that then we are done i.e. if B is NP-Complete and $B \leq PC$ for C in NP, then C is NP-Complete. Thus, we can verify that the **Subset Sum Problem** is NP-Complete using the following two propositions:
Subset Sum is in NP:
If any problem is in NP, then given a certificate, which is a solution to the problem and an instance of the problem (a set **S** of integer **a1…aN** and an integer **K**) we will be able to identify (whether the solution is correct or not) certificate in polynomial time. This can be done by checking that the sum of the integers in subset **S'** is equal to **K**.
Subset Sum is NP-Hard:
In order to prove Subset Sum is NP-Hard, perform a reduction from a known NP-Hard problem to this problem.
Carry out a reduction from which the **Vertex Cover Problem** can be reduced to the **Subset Sum problem**. Let us assume a graph G(V, E) where V = {1, 2, …, N}. Now, for every vertex i, **ai=i**. For every edge (i, j) we define a component called, **bij**.
We will represent the integers in a matrix format, where every row is expressed in the base-4 representation of the corresponding integer value of |E|+1 digits. The matrix has the following properties:

1. The first column contains an integer value 1 for **ai** and 0 for **bij**.

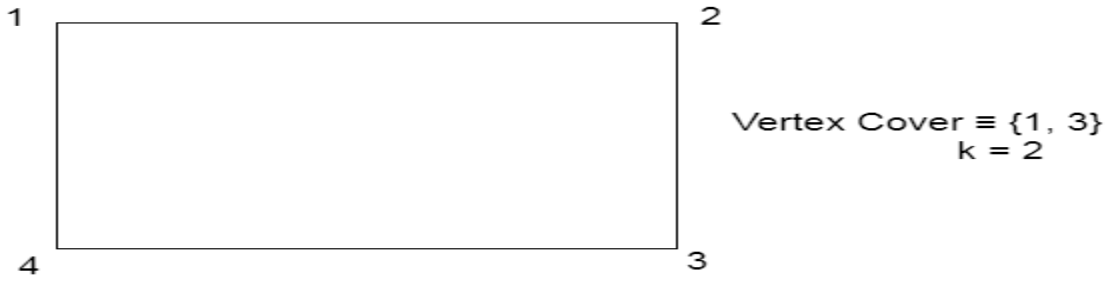
2. Each of the E columns starting from the right side of the matrix represents a digit for each edge. The column (i, j)=1 for **ai**, **aj** and **bij**, otherwise, it is equal to 0.
3. We define a constant k' such that,

Now, the following propositions hold:

- Let us consider a subset of vertices and edges to (V', E') respectively, such that

- **bij** can contain at most 1 in every column. Also, the k' parameter has a 2 in all less significant digits up to |E|. We can never have a carry-in these digits. Now, these digits sum up to at-most three 1's in each column. This implies that for every edge (i, j), V' must contain either i or j. Therefore, V' becomes a vertex cover.
- Let us assume there is a Vertex Cover of size k, we will choose integers **ai** such that i lies in V' and all **bij** Such that either i or j is in V'. On summation of all these integers in base 4 representation(that we choose from the matrix), we get sum of integers =k'. Therefore, the chosen integers form the subset of integers with sum = k'. Therefore, subset sum holds.

Let us consider the following example,
Given is a vertex cover V = {1, 3} and k = 2



Now, a1 = 1, a2 = 2, a3 = 3, a4 = 4
The matrix can be constructed in the following way:

Matrices M,

	e_{12}	e_{23}	e_{34}	e_{14}
a_1	1	1	0	0	1
a_2	1	1	1	0	0
a_3	1	0	1	1	0
a_4	1	0	0	1	1
b_{12}	0	1	0	0	0
b_{23}	0	0	1	0	0
b_{34}	0	0	0	1	0
b_{14}	0	0	0	0	1

Now, to prove the value of k' let us choose **ai** such that i lies in V', we choose a1 and a3 and **bij** such that either i or j lies in V', that is we choose **bij** such that either i or j is in V', that is we choose b12, b14, b23 and b34 from the matrix . In base 4 representation, we have the following values:
a1 = 321, a3 = 276, b12 = 64, b23 = 16, b14 = 1, b34 = 4
These values are computed using the matrix. On summation of these values, we get,
k' = 321 + 276 + 64 + 16 + 1 + 4 = 682.
Hence, k' value can be calculated and verified.
Therefore, the **Subset Sum Problem** is NP-Complete.

Subset sum

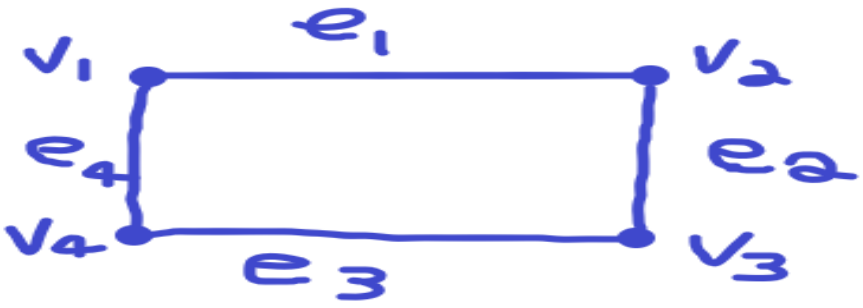
- **Input:** A multi set S of numbers and a natural number t
- **Question:** Does S contain a subset A such that $\sum x \in A x = t \sum$(e.g., {1,1,2,3,4,5}{1,1,2,3,4,5}, by multiset it means duplicates are allowed)

Vertex cover

- **Input:** An undirected graph G and a number k
- **Question:** Does G have a vertex cover of size k

Show: vertex cover ≤P≤subset sum

My work so far: I need to somehow transform (G,k)→(S,t)



Above is a example graph G that I constructed. It's vertex covers are {v1,v3}{1,3} so the size of the vertex cover is 22, thus k=2=2.
This is basically S
if ei and vj is incident I put 11; otherwise 00
Now what I need is a target t to satisfy this.
The matrix above can be written in a set to be S={1001,1100,0110,0011}={1001,1100,0110,0011} if I were to add up all these values probably in base 2 since its binary, it would give me a value but I think I need a consistent value t, so this wont work. I'm also unsure how to integrate the k=2=2 into this either.

For simplicity write all the numbers in base 44. Fix an arbitrary ordering of the edges of the graph G and let ei be the i-th edge (i=1,…,|E(G)|=1,…,|).
For each vertex v∈V(G)∈) define an integer nv of |E(G)|+1)|+1 digits as follows:

- the i-th least significant digit of nv is 11 if ei is incident to v and 00 otherwise;

If there is a subset A of S that sums to t, then A contains exactly k numbers from {nv:v∈V(G)}{:∈} (as otherwise the most significant digits of ∑x∈Ax∑∈ would not match those of t). Moreover, V′={vi:ni∈A}′={:∈} is a vertex cover for G.
To see this, pick any ei=(u,v)∈E(G)=(∈) and notice the i-th least significant digit of t is 22. Since, for j=1,…,|E(G)|=1,…,)|, there are exactly 33 numbers in S whose j-th least significant digit is non-zero (namely nu′, nv′, and mej, where ej=(u′,v′)=( ), we have that in the summation ∑x∈ Ax∑∈ no carry occurs among the |E(G)|least significant digits. We conclude that A contains exactly two numbers from {nu,nv,me}{}, implying that u∈ V′∈′ or v∈ V′∈′ (possibly both).

In your example graph, for k=3=3 you get the following (bold rows denote the elements of A when V′={v1,v3,v4}={1,3,4}):

Lecture 6

Applications of String Matching Algorithms

String matching algorithms have greatly influenced computer science and play an essential role in various real-world problems. It helps in performing time-efficient tasks in multiple domains. These algorithms are useful in the case of searching a string within another string. String matching is also used in the Database schema, Network systems.

Let us look at a few string matching algorithms before proceeding to their applications in real world. String Matching Algorithms can broadly be classified into two types of algorithms –

1. Exact String Matching Algorithms
2. Approximate String Matching Algorithms

Exact String Matching Algorithms:
Exact string matching algorithms is to find one, several, or all occurrences of a defined string (pattern) in a large string (text or sequences) such that each matching is perfect. All alphabets of patterns must be matched to corresponding matched subsequence. These are further classified into four categories:

1. **Algorithms based on character comparison:**
   - Naive Algorithm: It slides the pattern over text one by one and check for a match. If a match is found, then slides by 1 again to check for subsequent matches.
   - **KMP (Knuth Morris Pratt) Algorithm:** The idea is whenever a mismatch is detected, we already know some of the characters in the text of the next window. So, we take advantage of this information to avoid matching the characters that we know will anyway match.
   - **Boyer Moore Algorithm:** This algorithm uses best heuristics of Naive and KMP algorithm and starts matching from the last character of the pattern.
   - **Using the Trie data structure:** It is used as an efficient information retrieval data structure. It stores the keys in form of a balanced BST.

2. **Deterministic Finite Automaton (DFA) method:**
   - Automaton Matcher Algorithm: It starts from the first state of the automata and the first character of the text. At every step, it considers next character of text, and look for the next state in the built finite automata and move to a new state.

3. **Algorithms based on Bit (parallelism method):**
   - Aho-Corasick Algorithm: It finds all words in O(n + m + z) time where n is the length of text and m be the total number characters in all words and z is total number of occurrences of words in text. This algorithm forms the basis of the original Unix command fgrep.

4. **Hashing-string matching algorithms:**
   - Rabin Karp Algorithm: It matches the hash value of the pattern with the hash value of current substring of text, and if the hash values match then only it starts matching individual characters.

Approximate String Matching Algorithms:

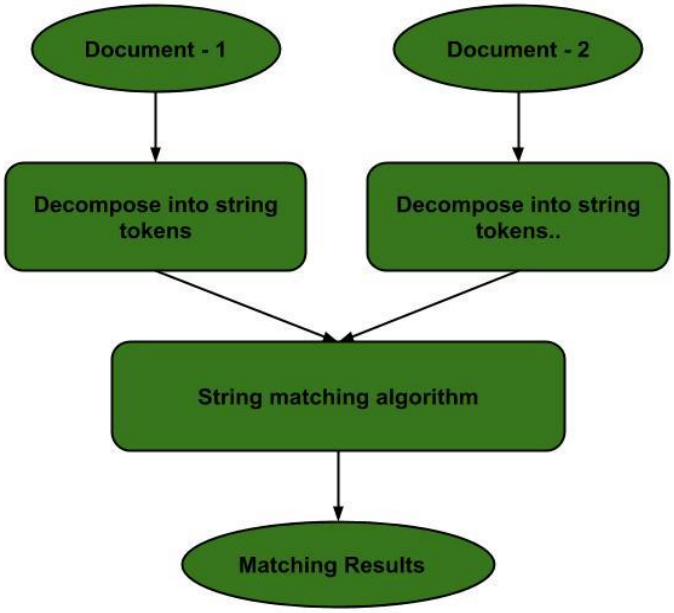
Approximate String Matching Algorithms (also known as Fuzzy String Searching) searches for substrings of the input string. More specifically, the approximate string matching approach is stated as follows: Suppose that we are given two strings, text T[1…n] and pattern P[1…m]. The task is to find all the occurrences of patterns in the text whose edit distance to the pattern is at most k. Some well known edit distances are – Levenshtein edit distance and Hamming edit distance.

These techniques are used when the quality of the text is low, there are spelling errors in the pattern or text, finding DNA subsequences after mutation, heterogeneous databases, etc. Some approximate string matching algorithms are:

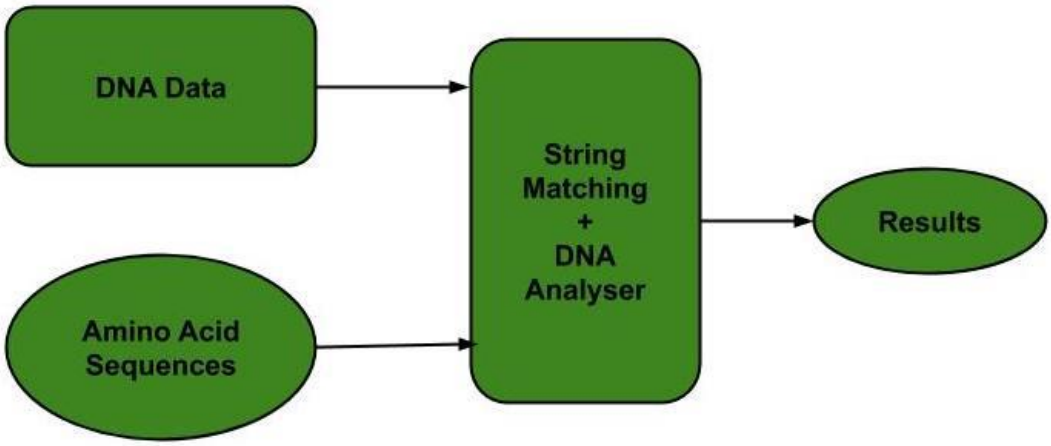
- **Naive Approach:** It slides the pattern over text one by one and check for approximate matches. If they are found, then slides by 1 again to check for subsequent approximate matches.
- Sellers Algorithm (Dynamic Programming)
- Shift or Algorithm (Bitmap Algorithm)

Applications of String Matching Algorithms:

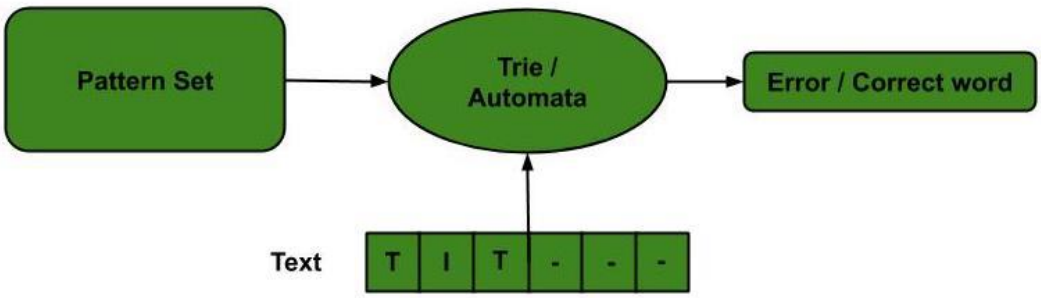
- **Plagiarism Detection:** The documents to be compared are decomposed into string tokens and compared using string matching algorithms. Thus, these algorithms are used to detect similarities between them and declare if the work is plagiarized or original.



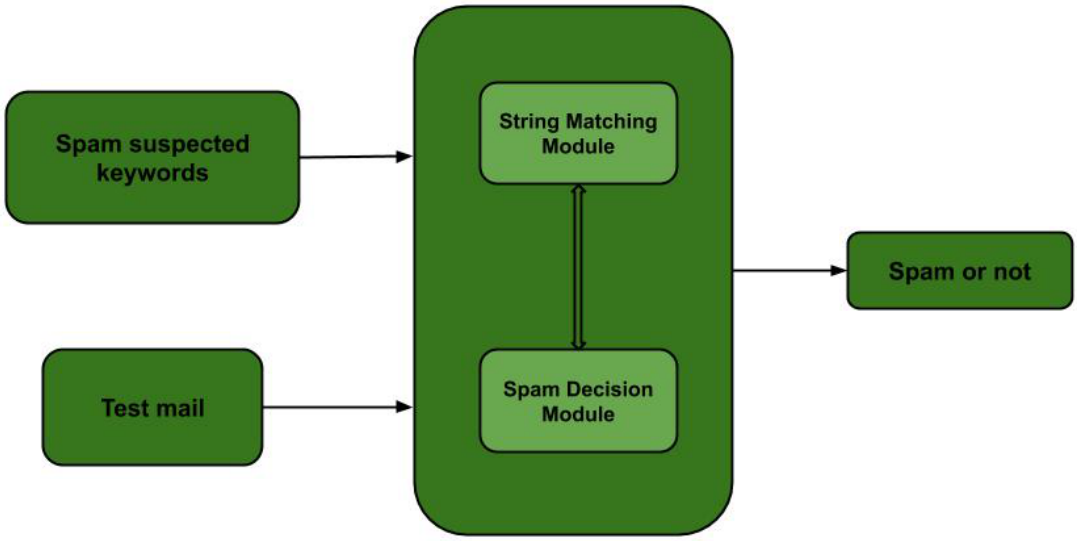
- **Bioinformatics and DNA Sequencing:** Bioinformatics involves applying information technology and computer science to problems involving genetic sequences to find DNA patterns. String matching algorithms and DNA analysis are both collectively used for finding the occurrence of the pattern set.



- **Digital Forensics:** String matching algorithms are used to locate specific text strings of interest in the digital forensic text, which are useful for the investigation.
- **Spelling Checker:** Trie is built based on a predefined set of patterns. Then, this trie is used for string matching. The text is taken as input, and if any such pattern occurs, it is shown by reaching the acceptance state.

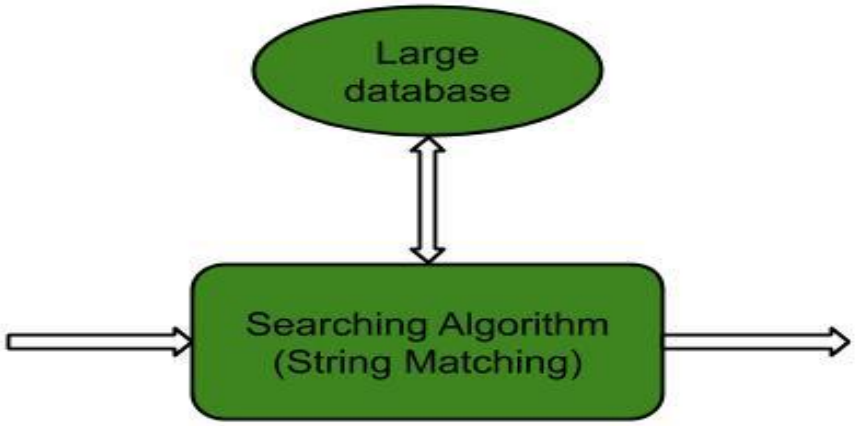


- **Spam filters:** Spam filters use string matching to discard the spam. For example, to categorize an email as spam or not, suspected spam keywords are searched in the content of the email by string matching algorithms. Hence, the content is classified as spam or not.

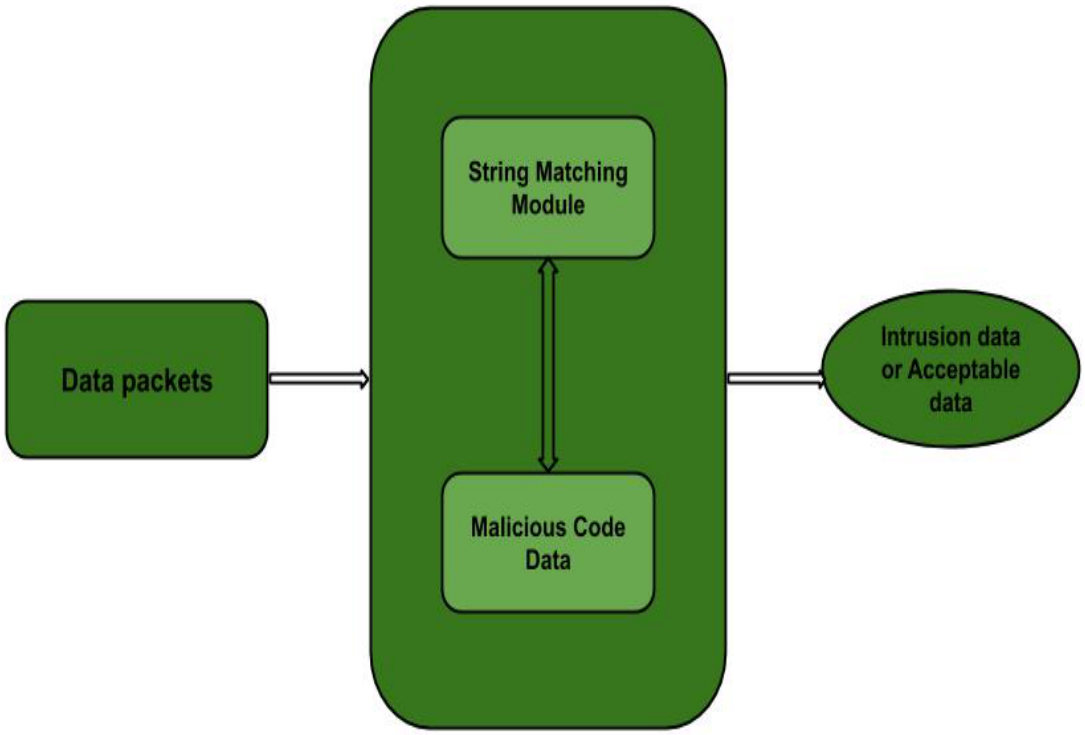


- **Search engines or content search in large databases:** To categorize and organize data efficiently, string matching algorithms are used. Categorization is done based on the search keywords. Thus, string matching

algorithms make it easier for one to find the information they are searching f



- **Intrusion Detection System:** The data packets containing intrusion-related keywords are found by applying string matching algorithms. All the malicious code is stored in the database, and every incoming data is compared with stored data. If a match is found, then the alarm is generated. It is based on exact string matching algorithms where each intruded packet must be detected.



Lecture 7

Approximation Algorithms

- **Overview:**

An approximation algorithm is a way of dealing with NP-completeness for an optimization problem. This technique does not guarantee the best solution. The goal of the approximation algorithm is to come as close as possible to the optimal solution in polynomial time. Such algorithms are called approximation algorithms or heuristic algorithms.

Features of Approximation Algorithm :

Here, we will discuss the features of the Approximation Algorithm as follows.

- An approximation algorithm guarantees to run in polynomial time though it does not guarantee the most effective solution.
- An approximation algorithm guarantees to seek out high accuracy and top quality solution(say within 1% of optimum)
- Approximation algorithms are used to get an answer near the (optimal) solution of an optimization problem in polynomial time

Performance Ratios for approximation algorithms :

Here, we will discuss the performance ratios of the Approximation Algorithm as follows.

Scenario-1 :

1. Suppose that we are working on an optimization problem in which each potential solution has a cost, and we wish to find a near-optimal solution. Depending on the problem, we may define an optimal solution as one with maximum possible cost or one with minimum possible cost,i.e, the problem can either be a maximization or minimization problem.
2. We say that an algorithm for a problem has an appropriate ratio of P(n) if, for any input size n, the cost C of the solution produced by the algorithm is within a factor of P(n) of the cost C* of an optimal solution as follows.

```
max(C/C*,C*/C)<=P(n)
```

Scenario-2 :

If an algorithm reaches an approximation ratio of P(n), then we call it a P(n)-approximation algorithm.

- For a maximization problem, 0< C < C*, and the ratio of C*/C gives the factor by which the cost of an optimal solution is larger than the cost of the approximate algorithm.

- For a minimization problem, 0< C* < C, and the ratio of C/C* gives the factor by which the cost of an approximate solution is larger than the cost of an optimal solution.

Some examples of the Approximation algorithm :

Here, we will discuss some examples of the Approximation Algorithm as follows.

1. **The Vertex Cover Problem –**

   In the vertex cover problem, the optimization problem is to find the vertex cover with the **fewest vertices**, and the approximation problem is to find the vertex cover with **few vertices**.

2. **Travelling Salesman Problem –**

   In the traveling salesperson problem, the optimization problem is to find the **shortest cycle**, and the approximation problem is to find a **short cycle**.

3. **The Set Covering Problem –**

   This is an optimization problem that models many problems that require resources to be allocated. Here, a logarithmic approximation ratio is used.

4. **The Subset Sum Problem –**

   In the Subset sum problem, the optimization problem is to find a subset of {x1,×2,×3…xn} whose sum is as large as possible but not larger than the target value t.